

Evaluation Design, Version 2

From Prompt to Autograded Exercise: Verified LLM Generation for Programming Education

An automated, comparative benchmark

Working document

July 17, 2026

This is an alternative to the first evaluation design. It narrows the scope to a single, sharper contribution: an *automated, reproducible benchmark* that shows how well the agentic approach generates auto-grader-ready introductory Java exercises and that objectively compares different large language models and approach variants. We focus on Java as use case for the automatically retrieved metrics. The guiding principle is to measure as much quality as possible *automatically and deterministically*, and to involve an expert *only* to validate an LLM-as-a-judge for the quality aspects that cannot be checked deterministically, never to author exercises. We organise measurement into three tiers: deterministic program analysis, an expert-validated LLM judge, and a small expert ground truth used solely for that validation. The judge is validated with a chunk-based inter-rater-agreement protocol borrowed from natural-language annotation, so expert time is bounded and the resulting benchmark scales to future models and approaches.

Contents

1. Purpose, scope, and what changed	2
2. Strategy: an automated, comparative benchmark	3
3. What to measure: a tiered quality model	3
4. The benchmark: dataset, factors, and protocol	5
5. Validating the LLM judge	6
6. Hypotheses	8
7. Analysis	8

8. Threats to validity	9
9. Feasibility and reproducibility	9
10. Open decisions to fix before running	10
A. Statistical methods at a glance	10

1. Purpose, scope, and what changed

Our approach contributes an agentic, human-in-the-loop method that generates the four mutually consistent artifacts of an Artemis programming exercise [1] from a short instructor brief, namely the problem statement, the template repository, the solution repository, and the test repository. A *verification oracle* is the sole acceptance gate: it rebuilds the candidate in fresh sandboxes and requires that the reference solution passes every gradable test and that the template compiles but fails all of them, plus integrity checks. An independent structured-output consistency check verifies cross-artifact claims and can be re-run fully automatically [2] but the consistency check is yet not part of the exercise generation pipeline.

Our evaluation focuses on demonstrating *how well the approach works* and to *objectively compare models and approaches*, expanding the automated evaluation as far as possible and involving experts only where automation is infeasible, and then only as evaluators of results rather than as authors of exercises. This plan proposes two concrete suggestions from the discussion: check as many quality properties as possible automatically and deterministically (does it compile, do the tests pass, do the required constructs appear), and, where a meaningful reference exists, compare the generated artifacts against a golden-truth reference drawn from existing course assignments.

Reframed research questions. The first design’s RQ1 is kept and expanded; its RQ2 and RQ3 are dropped from this paper and become future work.

RQ1 (quality)

How well does the agentic pipeline generate auto-grader-ready, mutually consistent, and didactically adequate introductory Java exercises, as measured automatically?

RQ2 (comparison)

How do different large language models and approach variants compare on these quality measures?

RQ3 (measurement)

How much of exercise quality can be assessed automatically and deterministically, and how well can an expert-validated LLM judge approximate expert judgement for the remainder?

RQ1 and RQ2 are the benchmark; RQ3 is the measurement methodology that makes the benchmark trustworthy and scalable. For this paper the approach variants are studied *without* an explicit competency taxonomy.

Scope of the human contribution. Experts never create exercises. A single expert (optionally a second for a reliability subset) rates a small calibration set of already-generated exercises, and this rating serves only to validate the LLM judge (Section 5). Everything else is automated and reproducible.

2. Strategy: an automated, comparative benchmark

We frame the study as a benchmark over many generated artifacts, the setting in which quantitative, comparative claims are strongest [3], and we structure the metrics with the Goal-Question-Metric paradigm [4]. Model-comparison benchmarks are the established way to evaluate code-generation systems: functional-correctness benchmarks such as HumanEval [5] and MBPP [6] report pass@k over many problems, and exercise-generation work already compares several models on shared metrics [7–9]. Our benchmark differs in that a unit is not a single function or exercise text but a complete, deployable, four-artifact exercise, so it needs a richer, exercise-specific metric set.

The design rests on three ideas:

1. **Measure automatically and deterministically wherever possible.** Compilation, test execution, the oracle criterion, coverage, mutation adequacy, the presence of required constructs, and cross-artifact consistency are all decidable by program analysis, with no model in the loop (Section 3).
2. **Use an expert-validated LLM judge only for the rest.** Didactical properties such as clarity or appropriate difficulty are not deterministically checkable; we score them with an LLM judge whose agreement with an expert has been established first (Section 5).
3. **Compare models and approaches on one fixed dataset.** A single golden-truth dataset and a fixed metric suite turn the evaluation into a reusable leaderboard for future models and approaches (Section 4).

3. What to measure: a tiered quality model

We separate measurement into three tiers by how the signal is obtained. Table 1 lists the concrete metrics.

Tier 1: deterministic program analysis (no model). These metrics are computed by building, running, and analysing the artifacts, and are fully reproducible. Buildability and grading come from the Artemis auto-grader, which runs the test repository against a submission and derives a score from the passed tests [1]; our verification oracle reuses this grading as its acceptance gate, requiring the reference solution to score 100 % and the template 0 %. Test-suite adequacy is measured by statement and branch *coverage* (JaCoCo) and by *mutation score* (PIT [24]), the standard way to show that a suite discriminates correct from incorrect code rather than merely passing [10,11]. This matters because weak tests mis-rank code-generation systems: strengthening a benchmark’s test suite alone can change which model appears best [25], so we treat each generated test

Table 1: Quality metrics for generated Artemis exercises, by tier. “Det.” marks fully deterministic metrics that need no model. The source column points to the method or the closest precedent.

Aspect	Metric / signal	How it is obtained	Det.	Src.
<i>Tier 1: deterministic program analysis</i>				
Buildability	template and solution compile	build in the Artemis sandbox	yes	[1]
Executability	test suite runs without harness error	JUnit runner	yes	[1]
Oracle correctness	solution 100 %, template 0 %	Artemis auto-grader [1]	yes	this work
Test coverage	statement and branch coverage	JaCoCo	yes	[10]
Test strength	mutation score (killed / generated)	PIT mutation testing	yes	[10, 11]
Concept fidelity	required constructs are present	AST queries, Checkstyle, PMD	yes	—
Complexity fit	cyclomatic / cognitive complexity in range	static analysis	yes	[12]
Consistency	residual cross-artifact inconsistencies	structured-output checker	yes	[2]
Efficiency	tokens, cost, latency, repair iterations	pipeline logging	yes	[8]
<i>Tier 1, reference-based: deviation from a golden-truth exercise</i>				
Code similarity	CodeBLEU, AST edit distance	compare solution to reference	yes	[13]
Behavioural match	golden tests vs. generated solution, and vice versa	differential testing	yes	[14]
Statement overlap	embedding similarity (weak signal)	compare statement to reference	yes	—
<i>Tier 2: expert-validated LLM judge (Section 5)</i>				
Clarity	rubric score	LLM judge, validated	no	[15, 16]
Solvability	rubric score	LLM judge, validated	no	[8]
Objective alignment	rubric score	LLM judge, validated	no	[17, 18]
Difficulty fit	rubric score, difficulty estimate	LLM judge + difficulty model	no	[19]
Scaffolding	rubric score	LLM judge, validated	no	[20]
Item-writing flaws	flaw checklist	rule-based check + LLM judge	part	[21, 22]
<i>Tier 3: expert ground truth</i>				
Expert rating	rubric score on calibration set	one expert (plus a reliability subset)	no	[23]

repository as an object to validate through coverage and mutation, not to trust, echoing prior study of LLM-generated test suites for introductory autograding [26]. Concept fidelity is checked structurally: we parse the reference solution and the tests into an abstract syntax tree and assert that the constructs implied by the target concept actually occur, for example a loop when the concept is iteration, using AST queries and static-

analysis rules (JavaParser, Checkstyle, PMD), and we bound the reference solution’s complexity with standard measures such as cyclomatic complexity [12]. Cross-artifact consistency is quantified by re-running the structured-output consistency checker [2] and counting residual inconsistencies.

Tier 1, reference-based: deviation from golden truth. Where a curated reference exercise exists for an input (Section 4), we measure how far the generated artifacts deviate from it. For code we report reference-based similarity with CodeBLEU [13] and abstract-syntax-tree edit distance, and, more meaningfully, *behavioural* agreement via differential testing [14]: run the golden tests against the generated solution and the generated tests against the golden solution. This dual, behavioural check mirrors the reference-versus-brute-force verification used to build competitive-programming problem sets [27]. For the problem-statement text we report embedding similarity as a weak signal only, because n-gram and embedding overlap correlate poorly with quality; behavioural and structural measures are the trustworthy ones.

Tier 2: expert-validated LLM judge. Didactical properties that resist deterministic checking, such as clarity, unambiguous specification, solvability from the statement alone, appropriate difficulty, alignment to the stated learning objective, and quality of scaffolding, are scored by an LLM judge on a compact rubric. These dimensions transfer methods from the automatic assessment of educational items in other domains: psychometrics characterises item quality through difficulty and discrimination [28]; the item-writing-flaw literature supplies validated taxonomies of what makes a question defective [21]; question-generation research defines multi-dimensional quality criteria such as clarity and answerability [16]; item difficulty can be estimated directly from item text [19]; and statement readability can be quantified with standard formulas [29]. Some of these checks are partly deterministic and belong in Tier 1: many item-writing flaws are caught more reliably by rule-based analysis than by an LLM judge [22], so we automate what rules can decide and reserve the judge for what genuinely needs judgement. LLM judges correlate well with human ratings on many text-quality tasks [30,31], but they carry position, verbosity, and self-preference biases [30,32]; we mitigate these (Section 5) and, crucially, we report the judge’s measured agreement with an expert rather than trusting it by default.

Tier 3: expert ground truth (validation only). A small expert-rated calibration set is the ground truth against which the Tier 2 judge is validated. It is the only human input in the design and involves rating, not authoring.

4. The benchmark: dataset, factors, and protocol

Dataset with golden truth. We assemble a small, curated dataset of **30 to 60** inputs derived from existing introductory course assignments, for example from the ITP and PSE courses, or courses at other universities (e.g., KIT). Each input is a short brief;

the corresponding vetted, deployed exercise serves as the golden-truth reference for the reference-based metrics. Using real course material keeps the dataset meaningful and gives a defensible notion of “good”. It also raises a data-contamination risk that we address in Section 8.

Factors. The benchmark crosses two factors. The first is the *model*, spanning open-weight and proprietary systems, for example **GPT-OSS**, **GPT-4.1-mini**, and **one stronger reference model**, all at a fixed decoding configuration. The second is the *approach variant*: at minimum a *single-pass* baseline that writes the four artifacts once, versus the *full agentic* pipeline with verify-and-repair and the oracle gate; optionally an intermediate variant with the loop but without the spec-fidelity critic, to separate repair from gating.

Runs. Because generation is stochastic, each (input, model, variant) cell is run k times. We report both an “any-of- k ” success rate in the spirit of pass@ k [5] and the per-run distribution, so that reliability, not only best case, is visible.

Output: a leaderboard. Results form a table of model $\times$ variant $\times$ metric, which is the reusable benchmark: a new model or approach can be dropped in and scored without re-running the human validation, as long as the rubric is unchanged.

5. Validating the LLM judge

The Tier 2 judge is only as trustworthy as its agreement with a human expert, so we establish that agreement before using the judge at scale. We adopt the calibration workflow standard in natural-language annotation, where annotators iterate on batches, measure agreement, adjudicate disagreements, and refine the guidelines until agreement is high enough to annotate the remainder independently [23, 33]. We treat the LLM judge as one annotator and an expert as the other.

Dimensions the judge scores. The judge rates the Tier 2 didactical dimensions of Table 1, each on a compact ordinal scale (met / partially met / not met): *clarity* (the problem statement is clearly written and unambiguous), *solvability* (the task can be solved from the statement alone), *objective alignment* (the exercise targets the intended concept or learning objective), *difficulty fit* (its difficulty suits the target level), *scaffolding* (starter code and hints give appropriate support), and the *item-writing flaws* that the Tier 1 rule-based checks cannot decide. Agreement is established and reported *per dimension*, because it varies across dimensions: only dimensions that clear the threshold are scored by the judge at scale, and the rest stay expert-only.

What the judge and expert see. Because Tier 1 already certifies the solution and tests mechanically (i.e., they compile, the solution passes, the template fails, coverage and mutation are adequate, and the required constructs are present), the didactical

Table 2: Artifacts the judge and expert must read to assess each didactical dimension (✓). The reference solution is never read for didactical rating: its correctness, use of the required constructs, and complexity are already certified deterministically in Tier 1 (Table 1). The raters are therefore shown the union of the marked columns, namely the problem statement, the template, and the tests.

Didactical dimension	Statement	Template	Tests	Solution
Clarity	✓			
Solvability	✓	✓		
Objective alignment	✓			
Difficulty fit	✓	✓		
Scaffolding	✓	✓		
Item-writing flaws	✓		✓	

assessment targets the *student-facing* artifacts. The judge and expert rate each exercise from the problem statement and the template, and additionally read the test repository for statement-consistency flaws, for example a worked example leaked into a graded test or a test that enforces a requirement the statement never states. The reference solution is provided as already-verified context rather than rated separately. Assessing the problem statement alone is insufficient: solvability and scaffolding require the template, and several item-writing flaws surface only when the statement is read against the tests. Table 2 maps each dimension to the artifacts it requires.

Protocol. We separate a small *calibration* phase, on which the judge may be tuned, from a *held-out validation* set, on which agreement is certified but the judge is never tuned; this keeps expert effort low and avoids fitting the prompt to the very items used to judge it.

1. Reserve a held-out validation set of **about 30** exercises, matching the double-coded subset size of Tutor Kai [20], that is never used for tuning.
2. *Calibrate*: on a small batch of **10 to 15** exercises, the expert and the judge score independently, we compute per-dimension agreement, adjudicate disagreements, and refine the judge’s rubric and prompt (the analogue of guideline refinement); one or two batches usually suffice.
3. *Freeze* the judge, then *validate*: the expert and the judge independently score the held-out set once.
4. Dimensions whose held-out agreement clears the pre-registered threshold are scored by the judge across the full corpus; the rest stay expert-only.

We report agreement with Gwet’s AC1 [34], which is robust when one rating category dominates, alongside Krippendorff’s α [35], with confidence intervals given the small sample, interpreted with the Landis and Koch bands [36] and the stricter 0.8 threshold conventional in computational-linguistics annotation [23]; the pre-registered target is “almost perfect” agreement, for example **AC1 ≥ 0.8** . Because our quality labels are prevalence-skewed, we report Gwet’s AC1 specifically to avoid the kappa paradox [37],

and, as a complementary bar, we check whether judge-expert agreement matches the expert-expert agreement measured on a reliability chunk [30]. Reference-guided grading, rubric decomposition, and order swapping with a consistency gate mitigate the judge’s known position, verbosity, and self-preference biases [30, 32].

What the validation licenses, and what it does not. Once a dimension clears the agreement threshold, the judge is used as a stand-in for the expert on that dimension across the full benchmark, and future models can be scored without new expert time. Dimensions that never reach the threshold are reported as expert-only and flagged as not yet automatable, rather than silently trusted; poor agreement on subjective dimensions such as difficulty is a known outcome and must be reported honestly [38]. Automated annotation with an LLM is trustworthy only when validated against human labels first [39, 40], and iteratively aligning an LLM evaluator’s rubric with expert judgement is an established human-in-the-loop practice [41].

6. Hypotheses

Thresholds in **blue** are to be fixed before running.

H1 (feasibility)

The full pipeline yields a deployable, oracle-passing exercise for at least **80 %** of inputs, and accepted exercises meet the Tier 1 thresholds: construct presence in **100 %**, median mutation score $\geq$ **0.8**, median coverage $\geq$ **0.9**, and no residual inconsistency in **90 %**.

H2a (approach)

The full agentic pipeline produces a higher share of deployable and consistent exercises than the single-pass baseline.

H2b (model)

Exercise quality differs across models, and the benchmark yields a stable ranking of the compared models.

H3 (measurement)

After chunk-based calibration, the LLM judge reaches the pre-registered agreement threshold with the expert on the core didactical dimensions, so it can score them at scale.

H4 (golden truth)

Higher-quality configurations deviate less from the golden references, that is, deviation from golden truth is associated with the automatic and judged quality scores.

7. Analysis

We report proportions with 95 % confidence intervals (Wilson) and per-metric distributions across the corpus. For H2a we compare configurations with McNemar’s or Fisher’s exact test on paired inputs and report an effect size. For H2b we compare models across

inputs with the Friedman test and a post-hoc pairwise procedure with a multiple-comparison correction, reporting rank-based effect sizes [42, 43]. For H4 we report rank correlations between golden-truth deviation and the quality scores. Throughout we lead with descriptive statistics and effect sizes and avoid over-claiming significance [44–46]. Agreement statistics for the judge are reported as in Section 5. Appendix A briefly defines each statistical test we use.

8. Threats to validity

We use the construct, internal, external, and conclusion categories [47].

Construct.

Automatic metrics may not fully capture “quality”. We mitigate with a broad, multi-faceted metric suite, with reference-based behavioural checks against golden truth, and with the expert-validated judge for the didactical dimensions the deterministic metrics cannot reach.

Internal: data contamination.

Because inputs derive from existing, possibly public course assignments, a model may reproduce memorised solutions, inflating similarity and quality [5]. We mitigate by preferring non-public or recent institutional assignments, by date-gating inputs against model training cutoffs where feasible [48], by paraphrasing briefs, by reporting a novelty check against the reference and against web and training-corpus matches, and by treating reference similarity as one signal among many rather than the target.

Internal: judge bias.

Position, verbosity, and self-preference biases [30, 32] are mitigated by randomised order, reference-guided grading, and rubric decomposition, and are bounded by the reported expert agreement.

External.

Results are specific to introductory Java, to Artemis, to the chosen dataset, and to the model versions tested; we mitigate by releasing the benchmark so others can re-run it on new models and settings, and by drawing inputs from more than one course where possible.

Conclusion.

Small datasets and many model comparisons risk false positives; we use paired non-parametric tests with corrections, report effect sizes and confidence intervals, and pre-register thresholds and the judge-agreement target [44].

9. Feasibility and reproducibility

The human cost collapses to validating the judge: one expert rates a small calibration batch and a held-out validation set of **about 30** exercises, on the order of **40** exercises in total, plus an optional second rater on the held-out set for an expert-expert agreement baseline. Everything else, the whole model $\times$ variant $\times$ metric grid, is automated and can be re-run on new models without further expert time. We release the dataset, the

generation and evaluation harness, the deterministic metric scripts, the judge rubric and prompts with their measured agreement, and the anonymised expert ratings, so the benchmark is a reusable artifact rather than a one-off study, and so that a model or approach added later is directly comparable.

10. Open decisions to fix before running

- Dataset size and source (ITP, PSE, or held-out assignments) and the contamination-control strategy.
- The exact model set and decoding configuration, and the approach variants (whether to include the loop-without-critic variant).
- Number of runs k per cell and the pass@ k reporting.
- The Tier 2 rubric dimensions, the calibration and held-out validation set sizes, and the agreement threshold that licenses autonomous judging.
- Which difficulty-estimation and construct-presence tools to standardise on.
- All **blue** thresholds in the hypotheses.

A. Statistical methods at a glance

For reference, the tests named in Sections 5 and 7 are summarised below. All are standard, and non-parametric or exact, matching our small samples and skewed, ordinal data.

Wilson score interval

A confidence interval for a binomial proportion that stays valid for small n and near-0/1 rates, where the normal approximation fails [49]. Used for all reported rates (yield, deployment correctness, per-dimension *met* rate).

McNemar’s test

A test for *paired* binary outcomes: on the same inputs, it compares two configurations using only the discordant pairs, the cases where they disagree [50]. Used for H2a (full pipeline vs. single-pass on the same briefs).

Fisher’s exact test

An exact test of association in a 2×2 table, valid for small or sparse counts where the χ^2 approximation is unreliable [51]. An unpaired alternative for count comparisons.

Friedman test

A non-parametric, rank-based analogue of repeated-measures ANOVA that tests whether three or more related groups differ; here, whether the models differ across the shared inputs [52].

Nemenyi post-hoc test

The standard pairwise follow-up after a significant Friedman test, comparing every pair of models while controlling the family-wise error rate; recommended for comparing multiple methods over many cases [53]. Where individual p -values are combined we additionally apply Holm’s correction [54].

Cliff’s δ / Vargha–Delaney $\hat{A}_{12}$

Rank-based effect sizes that express how often one condition stochastically dominates

another; robust for ordinal data and small n [42, 43].

Spearman's ρ

A rank correlation used for H4, relating golden-truth deviation to the quality scores without assuming linearity or normality [55].

Gwet's AC1 / Krippendorff's α

Chance-corrected inter-rater agreement coefficients; AC1 stays robust when one rating category dominates (the kappa paradox), which is likely for skewed quality labels [34, 35, 37].

pass@k

The unbiased estimate of the probability that at least one of k sampled generations passes, standard in code-generation benchmarks [5].

References

- [1] S. Krusche and A. Seitz, “Artemis: An Automatic Assessment Management System for Interactive Learning,” in *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 284–289.
- [2] F. T. J. Dietrich, Y. Zhou, T. Wasner, S. Krusche, and M. Acosta, “LLM-Based Multi-Artifact Consistency Verification for Programming Exercise Quality Assurance,” in *Proceedings of the 25th Koli Calling International Conference on Computing Education Research*. Koli Finland: ACM, 11 2025, pp. 1–11.
- [3] S. Easterbrook, J. Singer, M.-A. Storey, and D. Damian, “Selecting empirical methods for software engineering research,” in *Guide to Advanced Empirical Software Engineering*. Springer, 2008, pp. 285–311.
- [4] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” in *Encyclopedia of Software Engineering*. Wiley, 1994.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan *et al.*, “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [6] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan *et al.*, “Program synthesis with large language models,” arXiv preprint arXiv:2108.07732, 2021.
- [7] G. Zeng, Q. Xue, and X. Lu, “Instruction Tuning with LLMs for Programming Exercise Generation,” in *Web Information Systems and Applications*. Singapore: Springer Nature Singapore, 2024.
- [8] M. H. Nguyen, V.-A. Pădurean, A. Gotovos, S. Tschitschek, and A. Singla, “Synthesizing High-Quality Programming Tasks with LLM-based Expert and Student Agents,” in *Artificial Intelligence in Education (AIED 2025)*. Springer, 2025, arXiv:2504.07655.

- [9] N. Meißner, S. Speth, and S. Becker, “Automated Programming Exercise Generation in the Era of Large Language Models,” in *2024 IEEE 36th International Conference on Software Engineering Education and Training (CSEE&T)*, 07 2024.
- [10] K. Zheng, Y. Shen, and Y. Tao, “Automatic Unit Test Generation for Programming Assignments Using Large Language Models,” in *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*, 2025, pp. 242–252.
- [11] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [12] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [13] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, “CodeBLEU: A method for automatic evaluation of code synthesis,” arXiv preprint arXiv:2009.10297, 2020.
- [14] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [15] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, “Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models,” in *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1*, ser. ICER ’22. New York, NY, USA: Association for Computing Machinery, 2022.
- [16] W. Fu, B. Wei, J. Hu, Z. Cai, and J. Liu, “QGEval: Benchmarking multi-dimensional evaluation for question generation,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 11 783–11 803.
- [17] A. Del Carpio Gutierrez, P. Denny, and A. Luxton-Reilly, “Evaluating Automatically Generated Contextualised Programming Exercises,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSE 2024. New York, NY, USA: Association for Computing Machinery, 2024.
- [18] M. Jordan, K. Ly, and A. G. Soosai Raj, “Need a Programming Exercise Generated in Your Native Language? ChatGPT’s Got Your Back: Automatic Generation of Non-English Programming Exercises Using OpenAI GPT-3.5,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSE 2024. New York, NY, USA: Association for Computing Machinery, 2024.
- [19] L. Benedetto, P. Cremonesi, A. Caines, P. Buttery, A. Cappelli, A. Giussani, and R. Turrin, “A survey on recent approaches to question difficulty estimation from text,” *ACM Computing Surveys*, vol. 55, no. 9, 2023.

- [20] S. Jacobs, H. Peters, S. Jaschke, and N. Kiesler, “Unlimited Practice Opportunities: Automated Generation of Comprehensive, Personalized Programming Tasks,” in *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2025. New York, NY, USA: Association for Computing Machinery, 2025.
- [21] T. M. Haladyna, S. M. Downing, and M. C. Rodriguez, “A review of multiple-choice item-writing guidelines for classroom assessment,” *Applied Measurement in Education*, vol. 15, no. 3, pp. 309–334, 2002.
- [22] S. Moore, H. A. Nguyen, T. Chen, and J. Stamper, “Assessing the quality of multiple-choice questions using GPT-4 and rule-based methods,” in *Responsive and Sustainable Educational Futures (EC-TEL 2023)*, ser. LNCS, vol. 14200. Springer, 2023.
- [23] R. Artstein and M. Poesio, “Survey article: Inter-coder agreement for computational linguistics,” *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008.
- [24] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, “PIT: A practical mutation testing tool for Java (demo),” in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*, 2016, pp. 449–452.
- [25] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [26] U. Alkafaween, I. Albluwi, and P. Denny, “Automating autograding: Large language models as test suite generators for introductory programming,” *Journal of Computer Assisted Learning*, vol. 41, no. 1, 2025.
- [27] S. Zhou, Z. Zheng, K. Liu, Z. Shen, Z. Cheng *et al.*, “AutoCode: LLMs as problem setters for competitive programming,” arXiv preprint arXiv:2510.12803, 2025.
- [28] S. E. Embretson and S. P. Reise, *Item Response Theory: Foundations for Psychologists and Social Scientists*, 2nd ed. Routledge, 2026.
- [29] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, “Derivation of new readability formulas for navy enlisted personnel,” *Naval Technical Training Command, Research Branch Report 8-75*, 1975.
- [30] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [31] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 2511–2522.

- [32] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, and Z. Sui, “Large language models are not fair evaluators,” arXiv preprint arXiv:2305.17926, 2023.
- [33] J. Pustejovsky and A. Stubbs, *Natural Language Annotation for Machine Learning*. O’Reilly Media, 2012.
- [34] K. L. Gwet, “Computing inter-rater reliability and its variance in the presence of high agreement,” *British Journal of Mathematical and Statistical Psychology*, vol. 61, no. 1, pp. 29–48, 2008.
- [35] A. F. Hayes and K. Krippendorff, “Answering the call for a standard reliability measure for coding data,” *Communication Methods and Measures*, vol. 1, no. 1, pp. 77–89, 2007.
- [36] J. R. Landis and G. G. Koch, “The measurement of observer agreement for categorical data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [37] A. R. Feinstein and D. V. Cicchetti, “High agreement but low kappa: I. the problems of two paradoxes,” *Journal of Clinical Epidemiology*, vol. 43, no. 6, pp. 543–549, 1990.
- [38] E. Logacheva *et al.*, “Evaluating Contextually Personalized Programming Exercises Created with Generative AI,” in *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1*, ser. ICER ’24. New York, NY, USA: Association for Computing Machinery, 2024.
- [39] N. Pangakis, S. Wolken, and N. Fasching, “Automated annotation with generative AI requires validation,” arXiv preprint arXiv:2306.00176, 2023.
- [40] F. Gilardi, M. Alizadeh, and M. Kubli, “ChatGPT outperforms crowd workers for text-annotation tasks,” *Proceedings of the National Academy of Sciences*, vol. 120, no. 30, p. e2305016120, 2023.
- [41] S. Shankar, J. D. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, “Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences,” in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2024.
- [42] N. Cliff, “Dominance statistics: Ordinal analyses to answer ordinal questions,” *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.
- [43] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [44] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.

- [45] B. Kitchenham, L. Madeyski, D. Budgen, J. Keung, P. Brereton, S. Charters, S. Gibbs, and A. Pohthong, “Robust statistical methods for empirical software engineering,” *Empirical Software Engineering*, vol. 22, no. 2, pp. 579–630, 2017.
- [46] R. L. Wasserstein and N. A. Lazar, “The ASA statement on p-values: Context, process, and purpose,” *The American Statistician*, vol. 70, no. 2, pp. 129–133, 2016.
- [47] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [48] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and contamination free evaluation of large language models for code,” arXiv preprint arXiv:2403.07974, 2024.
- [49] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [50] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [51] R. A. Fisher, *The Design of Experiments*. Edinburgh: Oliver and Boyd, 1935.
- [52] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [53] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [54] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [55] C. Spearman, “The proof and measurement of association between two things,” *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.